

```

Kernel: arch/arm/boot/Image is ready
Kernel: arch/arm/boot/zImage is ready
Image Name: Linux-2.6.37
Created: Thu May 5 16:59:28 2011
Image Type: ARM Linux Kernel Image
(uncompressed)
Data Size: 1575492 Bytes = 1538.57 kB
= 1.50 MB
Load Address: 00000000
Entry Point: 00000000
Image arch/arm/boot/Image is ready

```

After the compilation step, the *uImage* is ready. Check the file's properties:

```

# file arch/arm/boot/uImage
uImage: u-boot legacy uImage,
Linux-2.6.37, Linux/ARM, OS Kernel
Image (Not compressed), 1575492 bytes,
Thu May 5 17:11:30 2011, Load Address:
0x00000000, Entry Point: 0x00000000,
Header CRC: 0xFC6898D9, Data CRC:
0x5D0E1B70

```

Now test this image on QEMU; the result is shown in Figure 1:

```

# qemu-system-arm -M versatilepb -m
128M -kernel /home/manoj/Downloads/
linux-2.6.37/arch/arm/boot/uImage

```

The kernel will crash at the point where it searches for a root filesystem, which you didn't specify in the above command. The next task is to develop a dummy filesystem for your testing. It's very simple—develop a small test C program *hello.c*, and use it to build a small dummy filesystem:

```

#include<stdio.h>
int main(){
while(1){
printf("Hello Open World\n");

```

```

getchar();
}

```

The endless loop (*while(1)*) will print a message when the user presses a key. Compile this program for ARM, but compile it statically; as you are trying to create a small dummy filesystem, you will not use any library in it. In GCC, the *'-static'* option does this for you:

```

# arm-none-linux-gnueabi-gcc hello.c
-static -o hello

```

Use the output file to create a root filesystem. The command *'cpio'* is used for this purpose. Execute the following command:

```

# echo hello | cpio -o --format=newc >
rootfs
1269 blocks

```

Check the output file:

```

[root@nxit qemu]# file rootfs
rootfs: ASCII cpio archive (SVR4 with no
CRC)

```

You now have a dummy filesystem ready for testing with this command:

```

# qemu-system-arm -M versatilepb -m
128M -kernel /home/manoj/Downloads/
linux-2.6.37/arch/arm/boot/uImage -initrd
rootfs -append "root=/dev/ram rdinit=/
hello"

```

When the kernel boots, it mounts *rootfs* as its filesystem, and starts the hello program as *init*. So now you are able to run ARM programs, and boot the ARM kernel inside QEMU. The next step would be to use u-boot on QEMU. An array of testing is ahead of us, which we will cover in a forthcoming article. 

By: Manoj Kumar

The author is a freelance developer and trainer. He leads a team in Linux kernel programming, Linux administration, cluster computing, embedded systems and QT/GTK programming on Linux. You can reach him at manoj@nxitsolutions.com and read his blog, www.open4india.blogspot.com






RHCE / RHCSS Exam Centre
RHCE ON RHEL 6
RHCVA

MCTS / MCITP
Windows 2008 Server
RHCE & RHCSS
 Training By **RHCSS** Faculty
 Only at **AEM** in Kolkata

ORACLE
 Clustering
 Shell Scripting
 Solaris PHP-MySQL



STUDENTS PLACED IN
 Wipro, HP, IBM, TCS, Tripod, Comtel,
 Diadem, PCS, CMS, Redhat
 & many more...

South Kolkata :
 7/1 Jadunath Sarkar Road
 Ph. : (033) 32518779, 9330925622
 North Kolkata :
 2/80 Dum Dum Road
 Ph. : 9830051236, 9830075018
www.aemk.org